Smart Tracking System: A Microservices Architecture utilizing Docker Swarm

Authors: Hızır Çiçekdağ, Tuğba Köroğlu
Group Theme: Group 4 - Container and Container Management

Abstract

This paper presents the design, development, and deployment of the "Smart Tracking System," a distributed microservices-based application. The primary objective is to demonstrate the practical implementation of containerization and container management—the core theme of Group 4. The system is decomposed into three distinct microservices: Package Service, Tracking Service, and Notification Service, all communicating asynchronously via a RabbitMQ message broker. To ensure isolation, consistency, and scalability, all components, including individual MongoDB database instances, are containerized using Docker. Furthermore, the entire architecture is orchestrated using Docker Swarm, which provides advanced container management capabilities such as automated load balancing, seamless service discovery, and robust self-healing mechanisms. The findings illustrate how leveraging container management platforms effectively addresses the complexities of distributed system deployment.

1. Introduction

In modern software engineering, the transition from monolithic architectures to microservices has become a standard approach for building scalable and maintainable applications. A monolithic application bundles all functionalities into a single executable, which often leads to tightly coupled codebases, difficult scaling, and "single points of failure." In contrast, a microservices architecture divides the application into smaller, loosely coupled, and independently deployable services.
However, managing multiple independent services introduces new operational complexities. Deploying, scaling, and networking these services manually is highly inefficient. This project addresses these challenges by applying containerization and container orchestration technologies to a real-world scenario: a Smart Tracking application.

2. Background: Container and Container Management

As assigned to Group 4, this project focuses heavily on the concepts of containers and their management.

2.1 Containerization (Docker)

A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another. In this project, Docker was selected as the containerization platform. Unlike traditional virtual machines (VMs) that require a full guest operating system, Docker containers share the host system's kernel, making them lightweight and fast to start.

2.2 Container Management (Docker Swarm)

While Docker provides the means to create containers, a Container Management tool (Orchestrator) is required to manage them in a clustered environment. Docker Swarm was chosen for this project due to its native integration with the Docker ecosystem. Swarm provides several critical capabilities:

- **Self-Healing:** Automatically detects failed containers and restarts them to maintain the desired state.
- **Load Balancing:** Distributes incoming network traffic equally across available service replicas.
- **Service Discovery:** Allows containers to find and communicate with each other using internal DNS via an overlay network.

3. Development Details

3.1 System Architecture

The Smart Tracking System is composed of the following entities:

Service Name	Role & Functionality	Database
Package Service	Handles the creation and management of cargo packages. Emits events to RabbitMQ.	MongoDB (package_db)
Tracking Service	Listens for new packages and simulates real-time tracking status updates.	MongoDB (tracking_db)
Notification Service	Listens for delivery statuses and triggers user notifications.	MongoDB (notification_db)

Service Name	Role & Functionality	Database
RabbitMQ	Acts as the central Message Broker (AMQP) for asynchronous inter-service communication.	N/A

3.2 Technologies Used

- **Backend Framework:** Node.js
- **Database:** MongoDB v7.0
- **Message Broker:** RabbitMQ v3.13
- **Containerization:** Docker
- **Orchestration:** Docker Swarm

3.3 Deployment and Orchestration

The deployment was fully automated using a declarative docker-compose.yml file configured for Docker Swarm stack deployment. Key configurations include:

1. **Overlay Network:** A custom Swarm overlay network named microservice_network was created to allow secure, multi-host communication between the isolated containers.
2. **Environment Variables:** Hardcoded configurations (such as localhost) were eliminated. Services connect to databases and RabbitMQ using Swarm's internal DNS resolution.
3. **Deployment Execution:** The entire cluster was instantiated using a single management command:

```
docker stack deploy -c docker-compose.yml akilli_takip
```

4. Conclusion

This project successfully demonstrates the transition from developing independent microservices to managing them effectively in a distributed environment. By leveraging Docker Swarm, the Smart Tracking System achieves high availability, fault tolerance, and scalable container management, fulfilling all the requirements defined for Group 4.